SINGAPORE UNIVERSITY
OF SOCIAL SCIENCES

ICT 133
Structured Programming

Tutor-Marked Assignment

July 2024 Semester Presentation

TUTOR-MARKED ASSIGNMENT (TMA)

This assignment is worth **24%** of the final mark for **ICT133, Structured Programming.**

The cut-off date for this assignment is **Sunday, 13 October 2024, 2355hrs.**

Note to Students:

All TMAs must be submitted electronically through Canvas. No partial submission of TMA will be accepted unless otherwise specified.

Please refer to undergraduate student handbook for more information:
Combined UG Student Handbook January 2024 (suss.edu.sg)

Assignment Requirements:

- Do **NOT** define classes for this TMA.
- **Unless specified in the question**, **you CANNOT use packages not covered in this module, e.g., re, collections, numpy, pandas etc.**
- All functions must be documented. Provide sufficient comments to your code and ensure that your program adheres to good programming practices such as not using global variables.
- **Do not use the exit() function.**
- Failing to do so can incur a penalty of as much as 50% of the mark allotted.

Submission Details:

- Use the template word document provided - **SUSS_PI_No-FullName_TMA.docx.** Rename the file with your SUSS PI and full name join with "_TMA" e.g., "**SUSS_PI-TomTanKinMeng_TMA.docx**" (without the quotes).
- Include the following particulars on the first page of the word document, on separate lines: **Course Code, SUSS PI No., Your Name, Tutorial Group and Submission Date**.
- Copy and paste the source code of each program you write in **text** format. Submit screenshots for **only** output of your program, where applicable.
- If you submit the source code as a screenshot, your code will not be marked. That is, **screenshot code will be awarded zero mark.**
- Submit your solution in the form of a **single MS Word document. Do NOT submit as a pdf document**. You will be penalised if you fail to submit a word document.
- The word document must be submitted to your respective T group. Besides this, you are required to upload your source code to Vocareum.

Question 1 (20 marks)

This question covers materials in Seminar 1 and 2. Use and express selection structure for this question. **Functions, repetition, or collections are not necessary for this question**.

a) As part of the Assurance Package (AP), every Singaporean aged 21 years and above, will receive AP Cash payments depending on his/her income and property ownership.

Singaporeans aged 21 and above in reference year	Disbursement Period	Owns 0 to 1 property			Owns more than 1 property
		Assessable Income (AI)			
		Up to $34,000	More than $34,000, and up to $100,000	More than $100,000	
2025	Dec 2024	$600	$350	$200	$200

Table 1

Write a program that helps Singaporean find out the estimated AP Cash amount they may receive from the Government.

Your program will read in the user's birth year, number of properties owned, and assessable income. Assume user will enter valid input.

Based on Table 1, your program will determine and display the eligible amount of AP Cash (if qualify). Study the sample executions below carefully:

```
Sample 1:
Enter year of birth: 2004
Do you own more than 1 property? (Y/N): N
Assessable Income : 30000
You will receive $600 AP Cash

Sample 2:
Enter year of birth: 1970
Do you own more than 1 property? (Y/N): Y
You will receive $200 AP Cash

Sample 3:
Enter year of birth: 2005
You are not eligible for AP Cash

Sample 4:
Enter year of birth: 1965
Do you own more than 1 property? (Y/N): N
Assessable Income : 80000
You will receive $350 AP Cash
```

(10 marks)

b) On top of AP Cash, lower-income senior Singapore citizens will receive additional cash payments through the AP Seniors' Bonus. See Table 2 below.

Property Ownership		Owns 0 to 1 property			
Assessable Income (AI)		Up to $34,000			
Annual Value (AV) of home		Up to $21,000		More than $21,000, and up to $25,000	
Disbursement Period	Age of Singaporean in Disbursement Year	55 to 64	65 and above	55 to 64	65 and above
Feb 2025	2025	$250	$300	$200	

Table 2

Enhance your program in part (a), such that the Seniors' Bonus will be displayed (if qualify). Study Table 2 and the sample executions below carefully:

```
Sample 1:
Enter year of birth: 1960
Do you own more than 1 property? (Y/N): N
Assessable Income : 30000
Annual value of home: 15000
You will receive $600 AP Cash + $300 AP Seniors' Bonus

Sample 2:
Enter year of birth: 1970
Do you own more than 1 property? (Y/N): Y
You will receive $200 AP Cash

Sample 3:
Enter year of birth: 1965
Do you own more than 1 property? (Y/N): N
Assessable Income : 34000
Annual value of home: 25000
You will receive $350 AP Cash + $200 AP Seniors' Bonus

Sample 4:
Enter year of birth: 1965
Do you own more than 1 property? (Y/N): N
Assessable Income : 80000
You will receive $350 AP Cash
```

(10 marks)

For each part, submit and paste screenshots of at least FOUR program executions, with different inputs.

Question 2 (25 marks)

This question covers materials up to seminar 3. Demonstrate your understanding of functions, selection, and repetition structures. **You may explore the use of list or tuple, but no dictionary or set is allowed for this question.**

Read the entire question before attempting. You can submit part (a) to (d) together.

a) An anagram is a word formed by rearranging the letters of a different word, typically using all the original letters exactly once.

Some examples:

"dog" = "god"
"Evil" = "vile"
"cheater" = "teacher"
"Santa" = "satan"

Create a Python function that to check if the given 2 words are anagram.

isAnagram (word1: str, word2: str) -> bool

- This function returns True if both words have the same letters (ignoring case). Otherwise, return False.
- For example,
  - isAnagram (“santa”, “Satan”) returns True
  - isAnagram (“santa”, “satay”) returns False

(6 marks)

b) Create a Python function *countVowels (word: str) -> int* that returns the number of vowels appearing in the given parameter word. If there is no vowel, the function returns 0. Assume the given parameter word can be either uppercase or lowercase.

For example,

- countVowels (“santA”) returns 2
- countVowels (“SUSS”) returns 1
- countVowels (“shy”) returns 0

(6 marks)

c) Write a function *countRepeatingChar(word: str) -> int* that returns the highest number of the "repeating letter" in the given parameter word. Assume the given parameter word can be either uppercase or lowercase.

For example, the function returns these values with the following parameters:

- countRepeatingChar('assistants') → 4
- countRepeatingChar('That') → 2
- countRepeatingChar('business') → 3
- countRepeatingChar('count') → 1

(6 marks)

d) Making use of the functions created above, write a Python program that will prompt the user to enter 2 words, and display message indicating:

- if the 2nd word is NOT an "Anagram" of the 1st word; or
- if the 2nd word is an "Anagram" of the 1st word; or
- if the 2nd word is a "Super Anagram" of the 1st word – where the 2nd word is an "Anagram", and the number of vowels equals to the number of repeating characters

Your program should repeatedly ask the user for words, until the user enters "X" or "x" as the 1st word.

<u>Sample program run (for reference only):</u>

```
Enter 1st word: dot
Enter 2nd word: got
got is NOT an anagram of dot

Enter 1st word: Teacher
Enter 2nd word: Cheater
Cheater is an anagram of Teacher

Enter 1st word: santA
Enter 2nd word: Satan
santA is a super anagram of Satan

Enter 1st word: X
bye
```

(7 marks)

Question 3 (20 marks)

This question covers materials up to Seminar 4 and requires you to solve computational problems using structured programming. **Only list should be used for question 3(b).**

Read the entire question before attempting. You need to submit each part individually.

a) Write a function *factorial1(n: int) -> int* that takes in an integer *n* as parameter. The function computes and returns the factorial of the number, *n*, computed using the formula:

$$factorial1(n) = 1 \times 2 \times 3 \times \ldots \times (n-1) \times n$$

(6 marks)

b) Write another function *factorial2(n: int, lookupTable: list): int* that takes **TWO (2)** parameters:

- *n*, a number to compute factorial for, and
- *lookupTable*, a list which stores the factorials at the correct indexes, which are already computed for future retrieval.

The function returns the factorial of the number, *n*.

Consider the two cases in getting the factorial of *n*.

Case 1
The function checks that the lookup table already contains *n* values. The function then returns the factorial value stored in index *n-1*.

For example, suppose `lookupTable = [1,2,6]` and *n* is 3, then the function returns 6, the value stored in index *n-1*.

Case 2
The function checks that the lookup table does not contain *n* values. The function locates the last factorial, *k* in the lookup table, computes and stores the calculated factorials of *k+1* to *n* into the lookup table.

For example, suppose `lookupTable = [1,2,6]` and *n* is 5. As the `lookupTable` has less than 5 values, it performs the following steps:

- Find the last factorial number in `lookupTable`, which is 3 (length of `lookupTable`)
- Compute the factorial of 4 by multiplying factorial 3, obtained in the lookup table, and 4, i.e., 6×4. Append the factorial of 4 in `lookupTable` (into index 3).
- Compute the factorial of 5 by multiplying factorial 4, obtained in the lookup table, and 5, i.e., 24×5. Append the factorial of 5 in `lookupTable` (into index 4).

When the function completes execution and returns 120, two elements would have been added to *lookupTable*, resulting in: `[1,2,6,24,120]`

Test your factorial functions using the main() program below:

```
def main():
    lookUpTable = [1,2,6]

    n = int(input("Enter n to compute factorial: "))
    print("Factorial1 of {}: {}".format(n, factorial1(n)))
    print("Factorial2 of {}: {}".format(n, factorial2(n, lookUpTable)))
    print(lookUpTable)

main()
```

(14 marks)

Question 4 (35 marks)

This question covers materials up to seminar 4. **The data structure to use is list/nested-list. No dictionary or set is allowed for this question. Keep the program modular by defining necessary functions.**

Read both all parts before attempting the question. You can submit all parts as one program.

The "Treasure Hunt" game is played by 2 players using a square board. Players take turns to enter the coordinates of selected square (row, col) to uncover the treasures, until a player has found more than half of the "treasures". We will illustrate the game using a 5x5 square board.

	0	1	2	3	4
0	?	?	?	?	?
1	?	?	?	?	?
2	?	?	?	?	?
3	?	?	?	?	?
4	?	?	?	?	?

Employ structured programming to develop this game with the following playing rules:

- At start of the game, prompt for the following:
  - Two <u>unique</u> names representing the of 2 players
  - Size of the game board, <u>*n*</u> that must be odd and at least 5
- Number of treasures will be $(n^2-1)/2$
- You program (the computer) will randomly plants the treasures into the board. For example, the 5x5 board below is planted with 12 treasures, represented using *, while – represents empty slots.

	0	1	2	3	4
0	*	-	-	-	*
1	-	*	*	*	-
2	-	*	*	-	-
3	*	*	-	*	-
4	-	*	-	*	-

- Computer will randomly decide which player to start.
- Each player will take turns to uncover a square, represented by "?"
  - Your program must validate the square that the player wanted to uncover.

	0	1	2	3	4
0	*	?	-	?	*
1	?	?	*	?	-
2	-	?	*	-	-
3	?	?	?	?	-
4	?	?	?	*	-

  - In the above board, if a player enters 5,4 for his next square, it is invalid as 5,4 is not within the board.
  - In the above board, if a player enters 0,0 or 0,2 for his next square, they are invalid as both 0,0 and 0,2 are already opened.
- Game ends when
  - a player has accumulated more than half of the treasures; or
  - no player can uncover any more squares
- If the result is a tie, repeat the game until a winner can be determined.

Sample program executions are as follows:

	Comments
Enter player 1 name: **Alan** Enter player 2 name: **Jane** Size of game board: **5** New Board generated. Treasure planted... 0 1 2 3 4 0 ? ? ? ? ? 1 ? ? ? ? ? 2 ? ? ? ? ? 3 ? ? ? ? ? 4 ? ? ? ? ?	
Jane, pick your square: **0,6** Wrong coordinates!! Please re-enter	After displaying a new board, randomly decide who to start.
Jane, pick your square: **0,0** Is a hit!! Current Score: Jane(1) Alan(0) 0 1 2 3 4 0 * ? ? ? ? 1 ? ? ? ? ? 2 ? ? ? ? ? 3 ? ? ? ? ? 4 ? ? ? ? ?	Show current score after each pick.
Alan, pick your square: **0,0** Square opened!! Please re-enter	
Alan, pick your square: **3,2** No treasure there... Current Score: Jane(1) Alan(0) 0 1 2 3 4 0 * ? ? ? ? 1 ? ? ? ? ? 2 ? ? ? ? ? 3 ? ? - ? ? 4 ? ? ? ? ?	
... ... after many more guesses ... ... Current Score: Jane(6) Alan(5) 0 1 2 3 4 0 * - - - * 1 - ? * * - 2 - * * - - 3 * * - * ? 4 - * ? * -	
Jane, pick your square: **3,4** No treasure there... Current Score: Jane(6) Alan(5) 0 1 2 3 4 0 * - - - * 1 - ? * * - 2 - * * - -	

3 * * - * - 4 - * ? * - Alan, pick your square: 1,1 Is a hit!! 0 1 2 3 4 0 * - - - * 1 - * * * - 2 - * * - - 3 * * - * - 4 - * ? * -	
It's a tie!! Jane(6) Alan(6) Rematch... New Board generated. Treasure planted... 0 1 2 3 4 0 ? ? ? ? ? 1 ? ? ? ? ? 2 ? ? ? ? ? 3 ? ? ? ? ? 4 ? ? ? ? ?	After Alan's guess, there is no more treasure, and their score is tied. → rematch
Alan, pick your square: 1,1 Is a hit!! Current Score: Alan(1) Jane(0) 0 1 2 3 4 0 ? ? ? ? ? 1 ? * ? ? ? 2 ? ? ? ? ? 3 ? ? ? ? ? 4 ? ? ? ? ?	Display a new board for rematch, and randomly decide who to start.
Jane, pick your square: **3,4** No treasure there... Current Score: Alan(1) Jane(0) 0 1 2 3 4 0 ? ? ? ? ? 1 ? * ? ? ? 2 ? ? ? ? ? 3 ? ? ? ? - 4 ? ? ? ? ? ... ... after many more guesses ... ... Current Score: Alan(6) Jane(3) 0 1 2 3 4 0 * - ? * * 1 - ? ? * - 2 * * - * - 3 ? - * - ? 4 ? - ? * ?	
Alan, pick your square: 4,2 Is a hit!! 0 1 2 3 4 0 * - ? * * 1 - ? ? * -	After Alan's guess, he has found 7 treasures, which is > than half of the treasures

2 * * - * - 3 ? - * - ? 4 ? - * * ? Final score: Alan(7) Jane(3) Alan is the winner!!	Declare Alan as the winner.

Consider creating the following functions as building blocks for the "Treasure Hunt" game program:

- *getPlayers( ) -> list*
  - This function will prompt players to enter their names and returns a list representing TWO unique player names.
  - For example: ['Jack', 'Jim']

- *getNewBoard(size: int) -> list*
  - This function should create and return a nested list based on the given parameter size, representing the game board, with "treasures" planted

- *printBoard(board: list)*
  - This function displays the current game board, showing un-opened, empty and treasure squares with symbol '?', '-' and '*'.

- *validatePick(row: int, col: int, gameBoard: list)*
  - This function validates that the given row and col should be within the range of the game board and returns True if the given row and col reference an un-opened square in the game board.
  - Otherwise, prints an appropriate message and returns False.

You will need to code other functions to complete the program.

Paste screenshots of game play that covers these scenarios:
- Player entering wrong coordinates (wrong row/column etc.)
- Player selecting an opened square.
- The game finishes with a winner.
- The game repeats when the result is a tie.
- Game with a different game board size.

(35 marks)

---- END OF ASSIGNMENT ----